JAVA WEEK 4

Object-Oriented Programming (OOP)

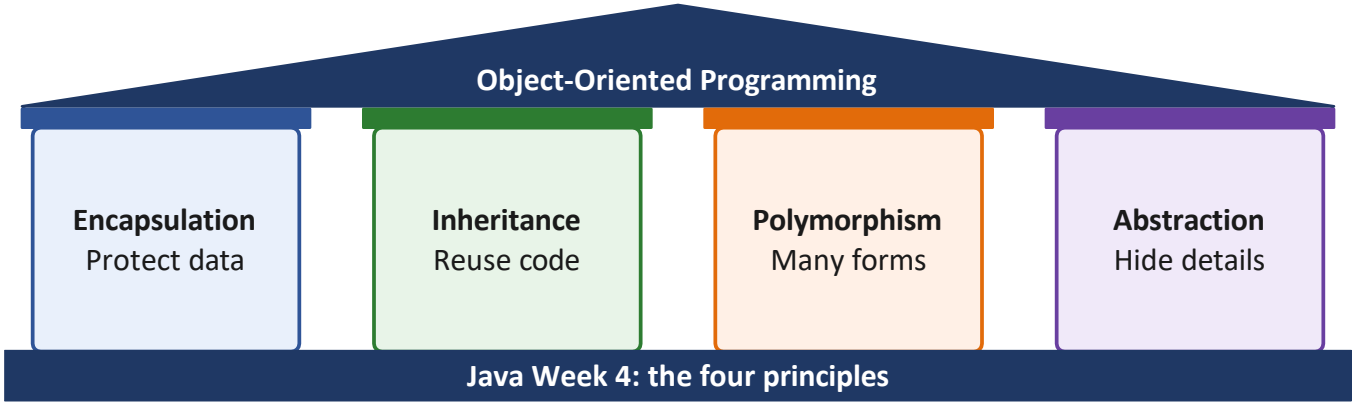


Figure 1: The four pillars of OOP covered this week

Encapsulation

Encapsulation means keeping data and the methods that work on that data together inside a class, while controlling how other parts of a program can access the data. A class protects its fields and exposes carefully chosen methods to read or update them.

In Java, encapsulation is commonly achieved by declaring fields **private** and providing **public** methods such as getters and setters. A setter can validate a value before storing it.

> **Real-life picture:** an ATM lets you press buttons (public methods), but you can never reach into the cash vault (private data). The machine checks every request first.

Access modifiers at a glance

Modifier	Same class	Same package	Subclass (other package)	Everywhere
private	**Yes**	**No**	**No**	**No**
default (no keyword)	**Yes**	**Yes**	**No**	**No**
protected	**Yes**	**Yes**	**Yes**	**No**
public	**Yes**	**Yes**	**Yes**	**Yes**

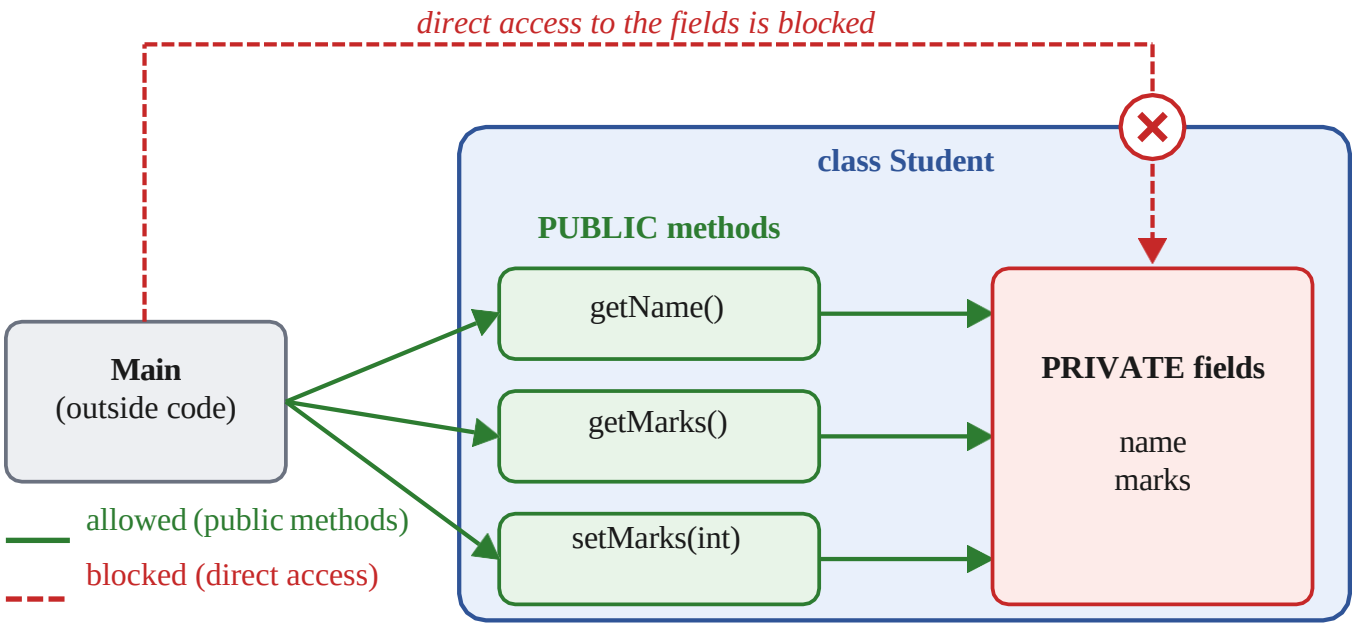


Figure 2: Outside code reaches the private fields only through public methods

Example: Student class

```
class Student {
    private String name;
    private int marks;

    public Student(String name, int marks){
        this.name= name;
        setMarks(marks);
    }

    public String getName(){ return name;}
    public int getMarks(){ return marks;}

    public void setMarks(int marks) {
        if (marks>= 0 && marks<= 100){
            this.marks= marks;
        } else {
            System.out.println("Invalid marks");
        }
    }
}
```

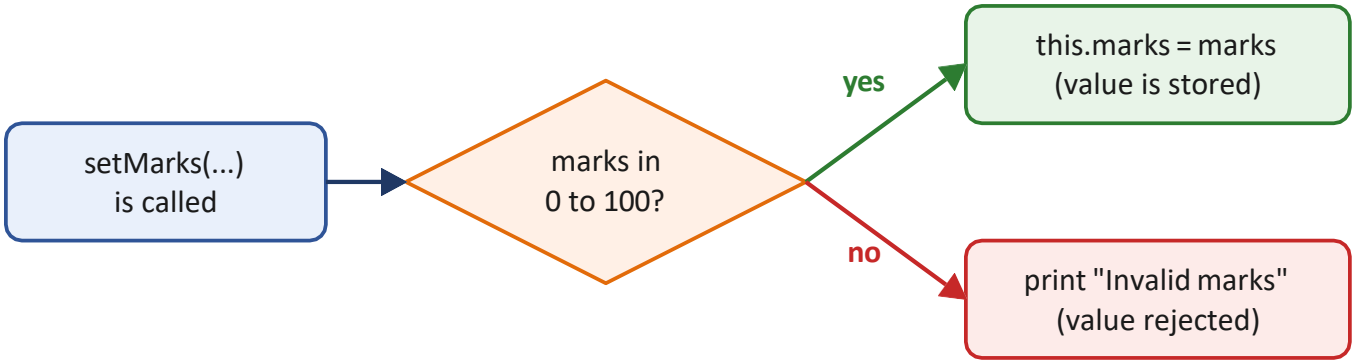


Figure 3: What the setter does with every new value

Using the class

```
public class Main {
    publicstaticvoidmain(String[]args){
        Students = newStudent("Riya",85);
        System.out.println(s.getName());
        System.out.println(s.getMarks());
        s.setMarks(110);
    }
}
```

Code explanation

- **private String name** and **private int marks** cannot be accessed directly from outside **Student**.
- **this.name = name** stores the supplied name in the current object; **this** refers to the current object.
- The constructor calls **setMarks()**, so validation also happens when the object is created.
- **getName()** and **getMarks()** return the private values. The setter only accepts marks from 0 to 100.

Output

```
Riya
85
Invalid marks
```

INHERITANCE

Reusing common behaviour through parent and child classes

Meaning

Inheritance allows one class to acquire accessible fields and methods from another class. It supports code reuse because shared behaviour can be written once in a parent class and reused by child classes. The existing class is the parent class or superclass; the inheriting class is the child class or subclass. Java uses **extends** for class inheritance.

Example: a Manager is an Employee. Employee can store shared details such as a name, while Manager adds manager-specific information such as a department.

Example code: Employee and Manager

```
class Employee{
    String name;

    Employee(Stringname){
        this.name= name;
    }

    void display(){
        System.out.println("Employee:" + name);
    }
}

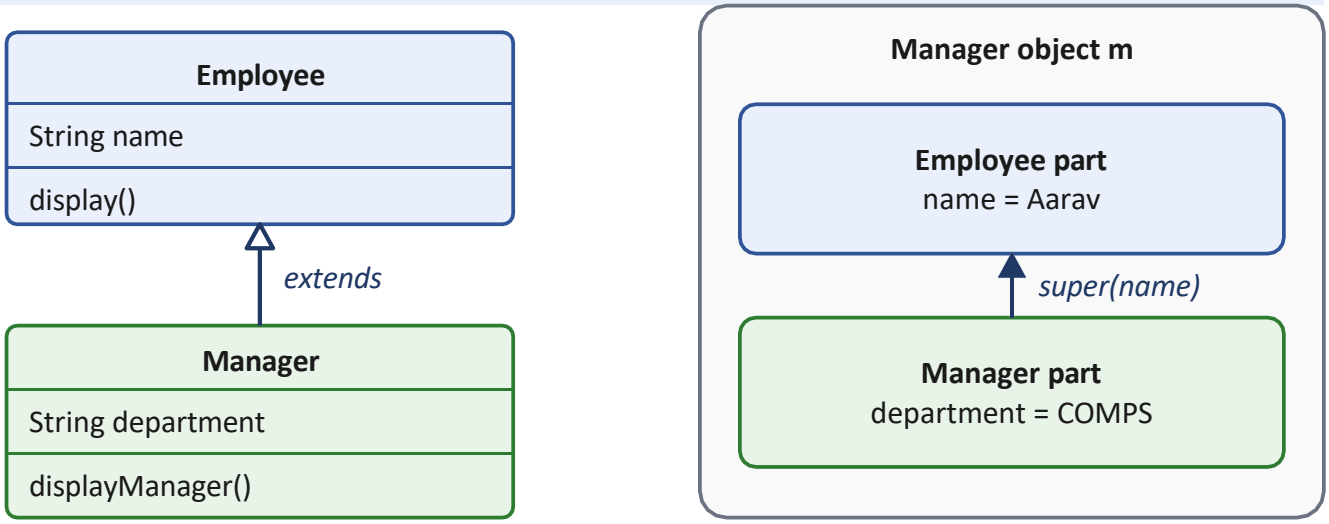
class Managerextends Employee{
    String department;

    Manager(Stringname,String department){
        super(name);
        this.department = department;
    }

    voiddisplayManager(){
        display();
        System.out.println("Department: " + department);
    }
}
```

```
public class Main {
    public static void main(String[]args) {
        Managerm = newManager("Aarav","COMPS");
        m.displayManager();
    }
}
```

Think "is-a": a Manager **is an** Employee, so Manager can do everything an Employee can, plus its own extra things.



*F*igure 4: Manager extends Employee, and a Manager object contains both parts

Code explanation

- **Employee** is the parent class; its constructor stores the name and **display()** prints it.
- **Manager** extends **Employee**, so it inherits accessible members such as **display()**.
- **super(name)** calls the parent constructor and passes the name. An explicit **super(...)** call must be the first statement in the child constructor.
- **this.department = department** stores the manager's department. The object call prints both the inherited name and the department.

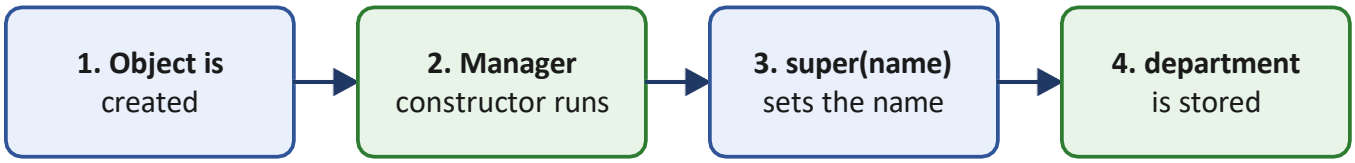


Figure 5: Order of events when new Manager("Aarav", "COMPS") runs

Output

```
Employee: Aarav
Department: COMPS
```

The super keyword

super refers to the parent-class part of the current object. It can:

- call a parent constructor using **super(...)**,
- call a parent method using **super.methodName()**,
- refer to an accessible parent field using **super.fieldName**.

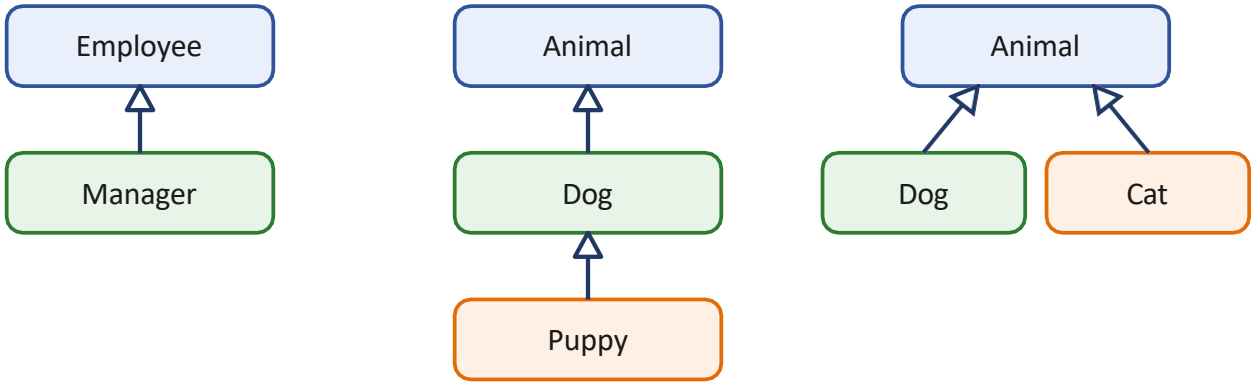
If a constructor does not explicitly call another constructor, Java automatically inserts **super()**. This works only when the parent has an accessible no-argument constructor; otherwise the code does not compile and you must call **super(...)** yourself.

INHERITANCE TYPES

Common class inheritance patterns in Java

Types of inheritance

Type	Meaning	Example
Single	One child class extends one parent class.	Manager extends Employee
Multilevel	A class extends a child class, forming a chain.	Puppy extends Dog; Dog extends Animal
Hierarchical	Multiple child classes extend the same parent.	Dog and Cat extend Animal



Single
one parent, one child

Multilevel
a chain of classes

Hierarchical
one parent, many children

Figure 6: The three class-inheritance patterns

Remember these terms

- **Parent class / superclass:** the class whose accessible members are inherited.
- **Child class / subclass:** the class that extends another class.
- **extends:** keyword used to inherit from a class.
- **super(...):** invokes a parent constructor and must appear first when written explicitly.
- **Code reuse:** shared fields and methods are defined once and reused by child classes.

Important Java rule

Java does not allow a class to extend two classes directly, so class-based multiple inheritance is not supported. However, one class can implement multiple interfaces.

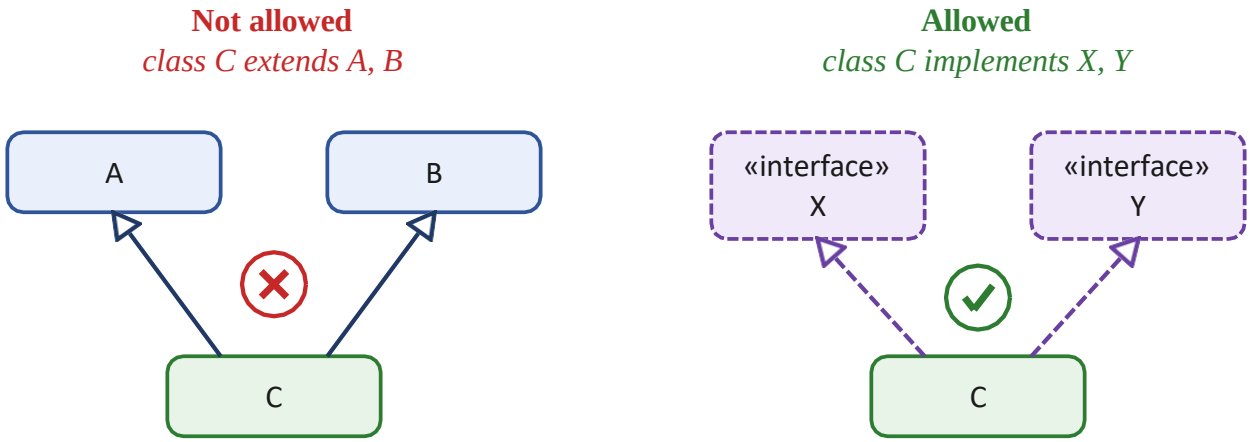


Figure 7: One parent class only, but as many interfaces as needed

Quick example

```
class Animal {
    void eat() { System.out.println("Eating"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}

// A Dog object can use both eat() and bark().
```

Here, Dog inherits **eat()** from Animal and defines its own **bark()** method. This illustrates how inheritance combines shared behaviour with specialised behaviour.

POLYMORPHISM

Method overriding and runtime method selection

Method overriding

Method overriding happens when a child class provides its own implementation of an inherited instance method with a compatible signature. It lets a child class specialise behaviour. The **@Override** annotation is recommended because it helps the compiler detect mistakes in the method signature.

```
class Payment {
    void pay() { System.out.println("Generic payment"); }
}

class CardPaymentextendsPayment{
    @Override
    void pay() { System.out.println("Paying by card"); }
}

class UpiPaymentextendsPayment{
    @Override
    void pay() { System.out.println("Paying by UPI"); }
}
```

```
public class Main {
    public static void main(String[]args) {
        Paymentp1 = newCardPayment();
        Paymentp2 = newUpiPayment();
        p1.pay();
        p2.pay();
    }
}
```

Code explanation

- **Payment p1 = new CardPayment()** means a variable declared as **Payment** refers to a **CardPayment** object.
- When **p1.pay()** runs, Java calls the overridden method belonging to the actual object.
- The same happens for **UpiPayment**. Choosing the implementation at runtime is called **dynamic method dispatch** or **runtime polymorphism**.

Output

```
Paying by card
Paying by UPI
```

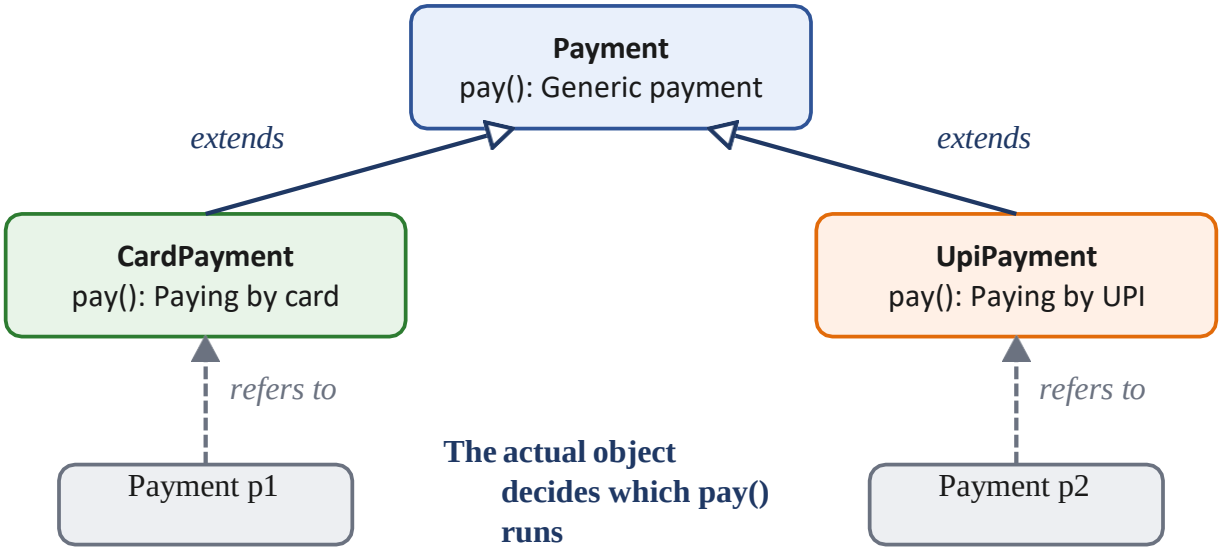


Figure 8: The variable type is Payment, but the object decides which pay() runs

Overloading vs overriding

Key	Method overloading	Method overriding
Meaning	Same method name with different parameters.	Child provides its own version of a parent method.
Classes	Usually in the same class.	Requires a parent and child class.
Parameters	Must differ in number, type, or order.	Same parameter list.
Return type	Changing only return type is not enough.	Same or compatible covariant type.
Selection	Compile time.	Runtime for overridden instance methods.



Figure 9: When Java decides which method to run

ABSTRACTION

Expose essential operations and hide implementation details

Meaning

Abstraction means showing essential operations while hiding unnecessary implementation details. When driving a car, a person uses the steering wheel, accelerator, and brake without needing to understand every internal detail of the engine. In Java, abstract classes and interfaces are commonly used to achieve abstraction.

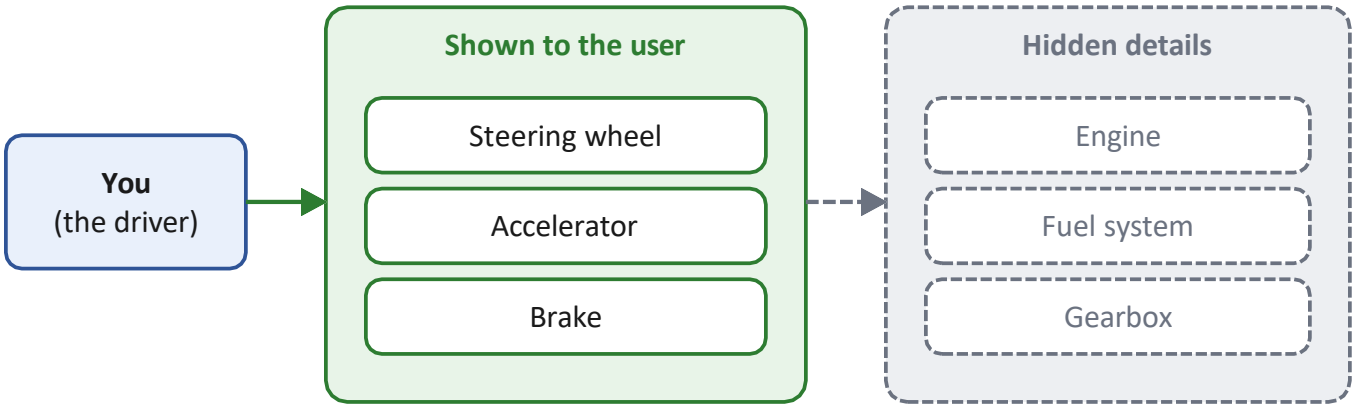


Figure 10: The driver uses the essential controls; the engine details stay hidden

Abstract classes

An abstract class is declared with the keyword **abstract** and cannot be instantiated directly. It can contain abstract methods, which have no method body, and concrete methods, which do have a body. A concrete child class must implement inherited abstract methods unless that child is also abstract.

```
abstract class Vehicle{
    abstract void start();

    void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car extends Vehicle {
    @Override
    void start() {
        System.out.println("Car starts with a key");
    }
}
```

```
public class Main {
    public static void main(String[] args){
        Vehicle v = new Car();
        v.start();
        v.stop();
    }
}
```

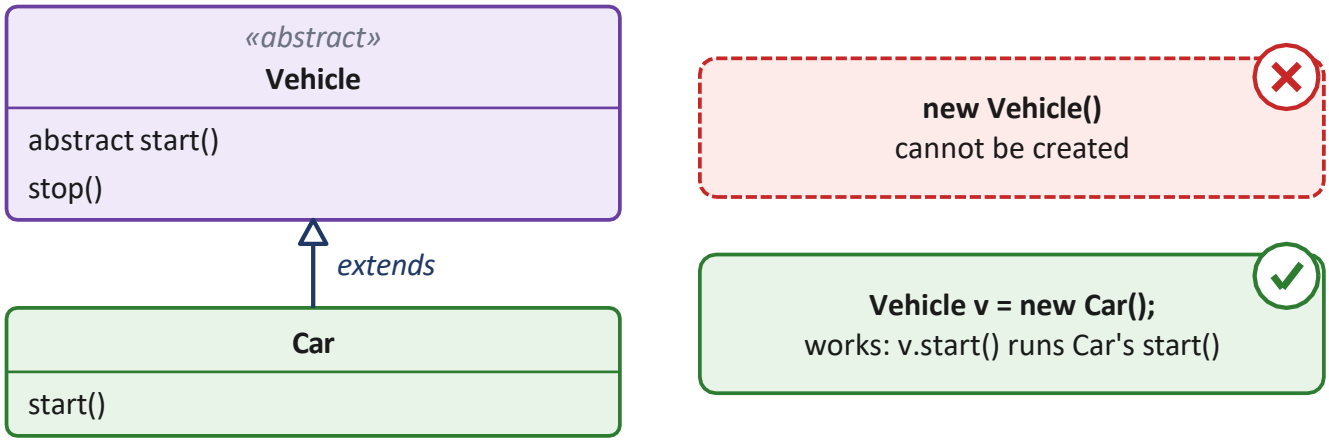


Figure 11: An abstract class is a partial blueprint; only concrete children become objects

Code explanation

- **abstract void start()** declares an operation without providing its implementation.
- **Car** extends **Vehicle** and provides the **start()** method.

- **stop()** already has a body in **Vehicle**, so **Car** inherits it.
- **Vehicle v = new Car()** creates a **Car** object referred to by a **Vehicle** variable.

Output

```
Car starts with a key
Vehicle stopped
```

Interfaces

An interface describes a contract that implementing classes agree to follow. A class uses **implements** to implement an interface. A basic interface method has no body and is implicitly public and abstract; the implementing class must provide a public implementation. Interfaces can also contain default and static methods.

Abstract class vs interface

Key	Abstract class	Interface
Keyword	**extends** to inherit	**implements** to follow the contract
How many	A class can extend only one class.	A class can implement many interfaces.
Methods	Abstract and concrete methods.	Abstract by default; can also have default and static methods.
Fields	Any kind of field.	Constants only (public static final).
Constructor	Yes.	No.

INTERFACES & PRACTICE

Putting the four OOP principles together

Interface example: GPS

```
interface GPS {
    void navigate();
}

class CarGPS implements GPS {
    @Override
    public void navigate(){
        System.out.println("Navigation started");
    }
}

public class Main {
    public static void main(String[] args){
        GPS gps = new CarGPS();
        gps.navigate();
    }
}
```

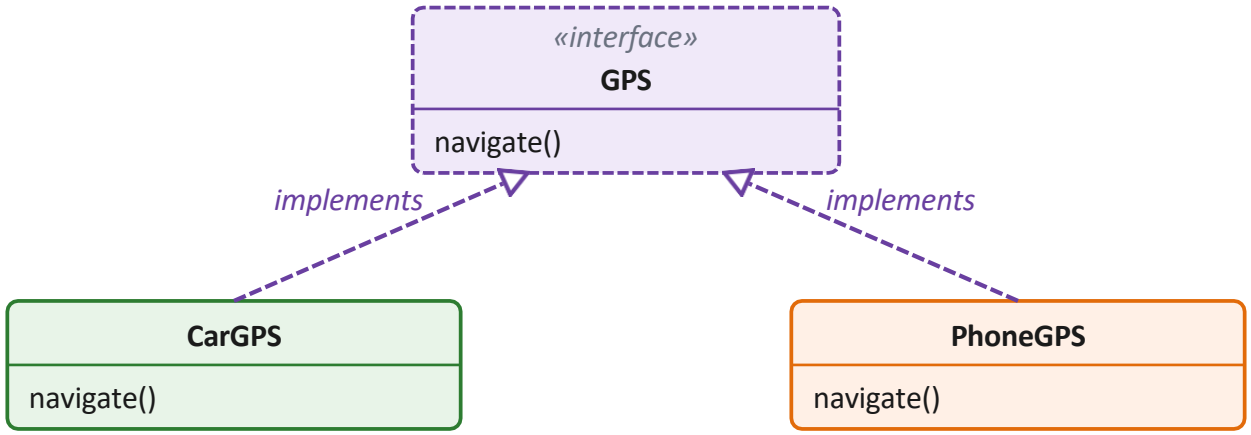


Figure 12: The interface is the contract; each class supplies its own version

Code explanation

- **GPS** declares the **navigate()** method.
- **CarGPS** implements **GPS** and supplies the method body.
- The implementation is **public** because the interface method is public.

- **GPS gps = new CarGPS()** means the variable has interface type **GPS** but refers to a **CarGPS** object.

Output

```
Navigation started
```

How the four OOP principles work together

Encapsulation protects an object's data. Inheritance allows a specialised class to reuse common code. Polymorphism lets a program call the correct implementation through a shared parent or interface type. Abstraction defines the essential operations that users should depend on.

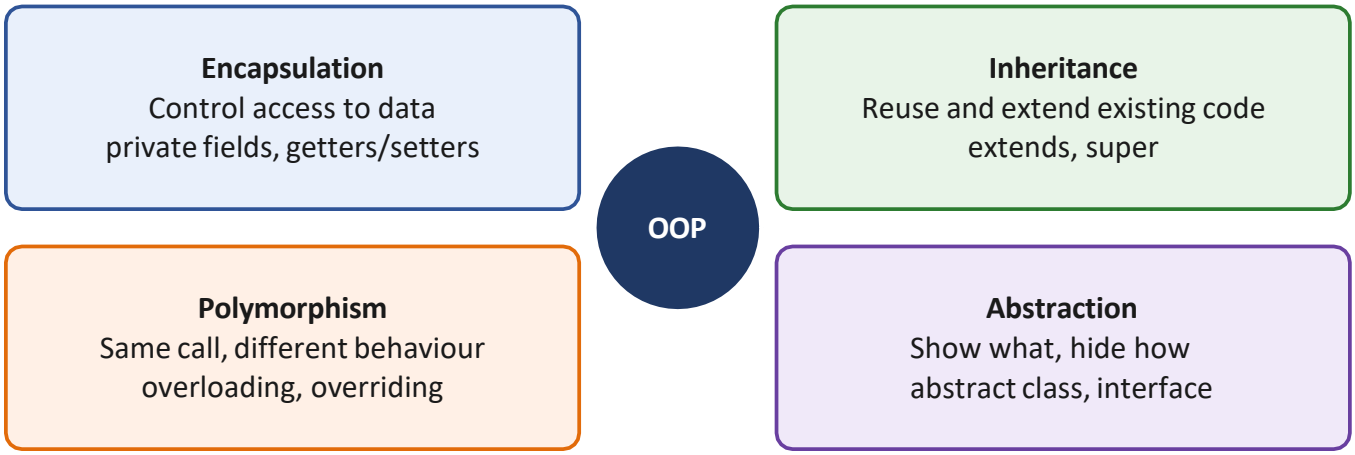


Figure 13: Each principle and the Java feature that delivers it

Example: a payment application may keep account details private, define a common Payment type, create CardPayment and UpiPayment subclasses, and call **pay()** without needing to know the internal steps of each payment method.

Practice questions

- **Encapsulation:** Create a BankAccount class with a private balance. Add **deposit()** and **withdraw()** methods. Reject negative deposits and withdrawals larger than the current balance. Add a getter to display the balance.
- **Inheritance:** Create a Person class with a name field and constructor. Create a Student class that extends Person and adds a roll number. Use **super()** to initialise the name and display both values.
- **Overloading:** Create a Calculator class with overloaded **multiply()** methods for two integers, three integers, and two double values.
- **Overriding:** Create an Animal class with **sound()**. Create Dog and Cat subclasses that override **sound()**. Use an Animal reference to call both versions.

- **Abstraction:** Create an abstract Appliance class with an abstract **turnOn()** method. Implement it in Fan and Light classes.
- **Interface:** Create a Payable interface and implement it in Employee and Freelancer classes. Call the required method through a Payable reference.

Final quick recap

Principle	Main idea	Typical Java feature
Encapsulation	Control access to data.	private fields, getters/setters
Inheritance	Reuse and extend existing code.	extends, super
Polymorphism	Same call can lead to different behaviour.	overloading, overriding
Abstraction	Show what an object does, hide how.	abstract class, interface

ONTO WEEK 5